

Bring Home the Treasure – Solution

This is a variant on the 0/1 Knapsack problem with traceback, where each algorithm proof stone tablet has:

- A one-word name (eg. A, B, C...etc)
- A certain weight; and
- A certain value.

Essentially, we're trying to find out what proofs Talia should put into her knapsack before heading home. The solution to this problem uses Dynamic Programming and keeps previous states saved in a DP table to prevent re-computing data.

Let:

- A: List of Algorithm proofs in (weight, value) format
- B: List of algorithm names for A, where A[i]'s algorithm name is B[i].
- n, w: the number of algorithm proofs and the maximum weight of the knapsack respectively.

Through a bottom-up approach, we:

1. Get the values of n, w, A, B from the user input.
2. Have a DP table of n+1 rows and w+1 columns, which each cell set as 0. This will hold our past states to prevent re-calculations. This can be implemented with a Python List.
3. From the top-left corner of the DP table, go through each state (all n, w in the table) until we reach the bottom right corner. For each state, choose whichever gets us the best value:
 - a. Put the item into the knapsack.
 - b. Don't put the item into the knapsack. If the item weight exceeds the current W we're at, we can't put the item in, so we'd have to go with this.

Then set the current state as that best value and continue. When we reach the bottom left corner, we'll get the best value we can get.

4. Next, we need to find the items that contribute to the best value. Starting from the bottom-right most corner, move left. When the current value `dpt[i][j]` differs from the value after it `dpt[i-1][j]`, the item at index i-1 must be in the knapsack since the value changed. Add that to the item list, move up so you're at j-1, and continue left. When you reach the start of the table again, you'll have a list of items that would make up the best value.
5. Return the best value (right-most bottom value of the DP table) and the item list in sentence format (a1, a2, a3, ...etc)

It is also possible by getting all possible subsets and finding the subset with the best value within the weight constraint, or recursing in to recalculate all states to get the best value, but this takes $O(2^n)$, so it is not the optimal solution. Some might also use fractional knapsack and a greedy algorithm to try to obtain the optimal solution. This will lead to the wrong answer since you can't exactly break apart proof tablets.

The best method would be through a bottom-up approach like above. A parent table can be used to keep track of items. Top-down will work but its implementation is more complex, especially regarding tracing back items.

Time Complexity

- Initial fetching of all proofs with their corresponding weights and values via `input()`: $O(n)$.
- We go through all possible states (1...n, 1...w) to find the best value and weight, by re-using solutions from previous n, w: $O(n*w)$.

- We only go through each state once: $O(1) * O(n*w) = O(n*w)$.
- Afterwards, we go through the states in the DP table, but decrement n until we reach 0. We traverse n times: $O(n)$.
 - Again, we only go through each state once.

So overall, our time complexity would be $O(n*w)$.

Space Complexity

- In the pseudocode example above, we have two lists: One that hold the algorithm proof's value and weight, and another holding their corresponding algorithm names: $O(n)$.
- n , w , r , c , weight, value, alg, etc all hold one value: $O(1)$.
- We have a DP table that has n number of rows and w number of columns that holds $n*w$ states: $O(n * w)$.
- The results set at most could have n number of algorithm proofs (all items are put into the knapsack): $O(n)$.

So overall, our space complexity would be $O(n * w)$. While we could shrink the memory by holding the last two rows of the DP table instead, we wouldn't be able to traceback to find the items.

Pseudocode (Example)

```
n, w, A, B <- get user input
dpt <- table of n+1 rows and w+1 columns
FOR LOOP:
Set counter i to 1
Break when counter reaches number n + 1
Increment counter by 1
  Set weight and value to A[counter - 1]
  FOR LOOP:
  Set second counter j to 1
  Break when second counter reaches number w + 1
  Increment second counter by 1
  IF second counter < weight:
    Set dpt, at row i and column j as dpt, at row i-1 and column j
  ELSE IF value in dpt, row i-1, column j > value at row i-1, column j - weight] +
    value
    Set dpt, at row i and column j as dpt, at row i-1 and column j
  ELSE
    Set dpt, at row i and column j as (dpt, at row i-1 and column j-weight) + value

result <- empty set
FOR LOOP:
Set counter k to n
Break when counter reaches 0
Increment counter by -1
  IF value in row i, column h, in dpt != value in row i-1, column h in dpt
    add name of item at index i-1 in B to result
    h <- h - weight of item at index i-1 in A

return (value in row -1, column -1 in dpt) and result
```

AI was not used for this project.